# 5. Backend Schema

**Project:** Advanced Personal Portfolio Website + Admin Panel **Database:** PostgreSQL (Supabase-compatible) — maps 1:1 to Firebase collections if needed **Version:** 1.0

> **Roman Urdu summary:** Yeh document database ke tables, columns, security rules (RLS), storage buckets, Edge Function APIs aur dummy (sample) data ka plan deta hai. Har table mein `is_sample` column hai taake admin baad mein dummy data ek click se replace/delete kar sake.

---

## 1. Conventions

- Primary keys: `uuid` (`gen_random_uuid()`), timestamps `timestamptz` (UTC), soft-visibility via `status`.
- Common columns on content tables: `id`, `created_at`, `updated_at`, `created_by`, `updated_by`, `is_sample boolean default false`, `sort_order int default 0`.
- `status` enum: `draft | scheduled | published | archived`. Public sees only `published` (or `scheduled` whose `publish_at <= now()`).
- Slugs: lowercase, hyphenated, unique per table.
- Multi-language readiness: text columns that need translation use a `*_i18n jsonb` companion (e.g. `{ "en": "...", "ur": "..." }`); English columns remain the source of truth.
- All tables have **RLS enabled**. No table is readable by anon unless a policy below says so.
- A trigger `set_updated_at()` runs before update; an audit trigger `log_audit()` writes to `audit_log` on insert/update/delete of content tables.

## 2. Entity Overview

```
site_settings (1) ── profile (1) ── social_links (n) ── nav_items (n) ──
home_sections (n)
intro_video (1) ── media_assets (n)
categories (n) ─◄ projects (n) ►── project_media (n)
                        │
                        ├── project_tags ► tags
                        ├── project_skills ► skills ► skill_groups
                        └── project_revisions
services (n) ── service_packages (n)      faqs (n)
experiences (n)  education (n)  certifications (n)  achievements (n)
testimonials (n)
blog_posts (n) ► blog_categories
inquiries (n) ── inquiry_notes (n)
seo_pages (n)
analytics_events (n)
admin_users (n)  audit_log (n)  rate_limits (n)
```

## 3. SQL — Types and Helper Functions

```
create extension if not exists pgcrypto;

create type content_status as enum
('draft','scheduled','published','archived');
create type inquiry_status as enum
('new','in_progress','replied','closed','spam');
create type admin_role as enum ('owner','editor');
create type media_kind as enum ('image','video','document','caption');

create or replace function set_updated_at() returns trigger language plpgsql
as $$
begin new.updated_at = now(); return new; end $$;

create table admin_users (
  user_id uuid primary key references auth.users(id) on delete cascade,
  role admin_role not null default 'editor',
  display_name text,
  created_at timestamptz not null default now()
);
```

```
create or replace function is_admin() returns boolean language sql stable
security definer set search_path = public as $$
  select exists (select 1 from admin_users where user_id = auth.uid());
$$;
create or replace function is_owner() returns boolean language sql stable
security definer set search_path = public as $$
  select exists (select 1 from admin_users where user_id = auth.uid() and
role = 'owner');
$$;

create or replace function is_live(s content_status, p timestamptz) returns
boolean language sql immutable as $$
  select s = 'published' or (s = 'scheduled' and p is not null and p <=
now());
$$;
```

## 4. SQL — Site-Level Tables (singletons and settings)

```
create table site_settings (
  id int primary key default 1 check (id = 1),           -- singleton
  site_name text not null default 'Your Name',
  tagline text,
  contact_email text,
  whatsapp_number text,
  booking_url text,
  availability_enabled boolean not null default true,
  availability_text text default 'Available for new projects',
  default_theme text not null default 'system' check (default_theme in
('system','dark','light')),
  accent_palette text not null default 'violet' check (accent_palette in
('violet','blue')),
  contact_success_message text default 'Thanks! I will reply as soon as
possible.',
  inquiry_retention_days int not null default 365,
  analytics_retention_days int not null default 730,
  feature_flags jsonb not null default '{

"intro_enabled":true,"testimonials_enabled":true,"faq_enabled":true,"pricing_
enabled":false,

"blog_enabled":false,"command_palette_enabled":true,"language_switcher_enable
```

```
d":false,

"analytics_enabled":true,"three_d_hero_enabled":false,"custom_cursor_enabled"
:false,

"scroll_progress_enabled":true,"hide_on_scroll_nav":false,"sample_banner_enab
led":false}'::jsonb,
  updated_at timestamptz not null default now(),
  updated_by uuid references auth.users(id)
);

create table profile (
  id int primary key default 1 check (id = 1),
  full_name text not null default 'Your Name',
  professional_title text not null default 'Your Professional Title',
  value_proposition text,
  bio_short text,
  bio_long text,                      -- sanitized HTML or markdown
  portrait_media_id uuid,             -- references media_assets(id)
  roles_rotating text[] default '{}', -- optional hero role rotation
  values jsonb default '[]',          -- [{title, text, icon}]
  languages_complete text[] default '{en}',
  is_sample boolean not null default false,
  updated_at timestamptz not null default now()
);

create table social_links (
  id uuid primary key default gen_random_uuid(),
  platform text not null,            -- github, linkedin, x, instagram,
youtube, behance, dribbble, whatsapp, email, other
  label text not null,
  url text not null check (url ~* '^(https://|mailto:|tel:)'),
  show_in_hero boolean default true,
  show_in_footer boolean default true,
  sort_order int default 0,
  is_sample boolean not null default false
);

create table nav_items (
  id uuid primary key default gen_random_uuid(),
  label text not null,
  target text not null,              -- '/#about', '/projects', '/resume'
  is_cta boolean default false,
```

```
  visible boolean default true,
  sort_order int default 0
);

create table home_sections (
  key text primary key,              -- hero, about, skills, projects,
services, experience, testimonials, faq, contact
  title text,
  eyebrow text,
  visible boolean default true,
  sort_order int default 0,
  settings jsonb default '{}'
);

create table intro_video (
  id int primary key default 1 check (id = 1),
  enabled boolean not null default true,
  video_desktop_id uuid, video_mobile_id uuid, poster_id uuid,
poster_mobile_id uuid, captions_en_id uuid, captions_ur_id uuid,
  focal_x numeric(5,2) default 50, focal_y numeric(5,2) default 35,    --
object-position %
  end_color text default '#0A0B10',
  load_timeout_seconds int default 6 check (load_timeout_seconds between 2
and 20),
  autoplay_muted boolean default true,
  script_en text, script_ur text,
  show_name_overlay boolean default true,
  is_sample boolean not null default false,
  updated_at timestamptz not null default now()
);
```

## 5. SQL — Media

```
create table media_assets (
  id uuid primary key default gen_random_uuid(),
  kind media_kind not null,
  bucket text not null,              -- media | intro | documents
  path text not null,
  public_url text not null,
  mime_type text not null,
  size_bytes bigint not null,
  width int, height int, duration_seconds numeric,
  alt_text text,                     -- required for images unless decorative
  decorative boolean default false,
  title text,
  version int not null default 1,    -- cache busting on replace
  created_at timestamptz not null default now(),
  created_by uuid references auth.users(id),
  is_sample boolean not null default false,
  constraint alt_required check (kind <> 'image' or decorative or
coalesce(alt_text,'') <> '')
);
create unique index on media_assets (bucket, path);
```

## 6. SQL — Projects and Taxonomy

```
create table categories (
  id uuid primary key default gen_random_uuid(),
  name text not null, slug text not null unique,
  sort_order int default 0, is_sample boolean not null default false
);

create table tags (
  id uuid primary key default gen_random_uuid(),
  name text not null unique, slug text not null unique,
  is_sample boolean not null default false
);

create table projects (
  id uuid primary key default gen_random_uuid(),
  slug text not null unique check (slug ~ '^[a-z0-9]+(-[a-z0-9]+)*$'),
  title text not null,
```

```
  summary text not null,                        -- short description (card)
  category_id uuid references categories(id) on delete set null,
  cover_media_id uuid references media_assets(id) on delete set null,
  demo_video_url text check (demo_video_url is null or demo_video_url ~*
'^https://'),
  problem text, approach text, role text, process text, challenges text,
results text,   -- sanitized rich text
  tools text[] default '{}',
  project_date date,
  live_url text check (live_url is null or live_url ~* '^https://'),
  source_url text check (source_url is null or source_url ~* '^https://'),
  featured boolean not null default false,
  status content_status not null default 'draft',
  publish_at timestamptz,
  seo_title text, seo_description text, og_media_id uuid references
media_assets(id) on delete set null,
  sort_order int default 0,
  view_count bigint not null default 0,
  is_sample boolean not null default false,
  created_at timestamptz not null default now(), updated_at timestamptz not
null default now(),
  created_by uuid references auth.users(id), updated_by uuid references
auth.users(id)
);
create index projects_status_idx on projects (status, publish_at);
create index projects_featured_idx on projects (featured, sort_order);
create index projects_search_idx on projects using gin (to_tsvector('simple',
coalesce(title,'')||' '||coalesce(summary,'')||' '||array_to_string(tools,'
')));

create table project_media (
  id uuid primary key default gen_random_uuid(),
  project_id uuid not null references projects(id) on delete cascade,
  media_id uuid not null references media_assets(id) on delete restrict,
  caption text, sort_order int default 0
);

create table project_tags (project_id uuid references projects(id) on delete
cascade, tag_id uuid references tags(id) on delete cascade, primary key
(project_id, tag_id));

create table project_revisions (
  id uuid primary key default gen_random_uuid(),
```

```
  project_id uuid not null references projects(id) on delete cascade,
  snapshot jsonb not null, note text,
  created_at timestamptz not null default now(), created_by uuid references
auth.users(id)
);
```

## 7. SQL — Skills, Experience, Services, Trust

```
create table skill_groups (
  id uuid primary key default gen_random_uuid(),
  name text not null, slug text not null unique, icon text,
  sort_order int default 0, is_sample boolean not null default false
);
create table skills (
  id uuid primary key default gen_random_uuid(),
  group_id uuid references skill_groups(id) on delete cascade,
  name text not null, context text,                    -- short real-world
context, no percentages
  is_tool boolean not null default false, logo_media_id uuid references
media_assets(id) on delete set null,
  visible boolean default true, sort_order int default 0, is_sample boolean
not null default false
);
create table project_skills (project_id uuid references projects(id) on
delete cascade, skill_id uuid references skills(id) on delete cascade,
primary key (project_id, skill_id));

create table experiences (
  id uuid primary key default gen_random_uuid(),
  role text not null, organization text, location text,
  start_date date not null, end_date date, is_current boolean default false,
  description text, highlights text[] default '{}',
  visible boolean default true, sort_order int default 0, is_sample boolean
not null default false
);
create table education (
  id uuid primary key default gen_random_uuid(),
  institution text not null, degree text, field text, start_year int,
end_year int, notes text,
  visible boolean default true, sort_order int default 0, is_sample boolean
not null default false
```

```
);
create table certifications (
  id uuid primary key default gen_random_uuid(),
  title text not null, issuer text, issued_on date, credential_url text check
(credential_url is null or credential_url ~* '^https://'),
  visible boolean default true, sort_order int default 0, is_sample boolean
not null default false
);
create table achievements (
  id uuid primary key default gen_random_uuid(),
  title text not null, description text, year int, metric_value numeric,
metric_label text,   -- metric only if verified
  proof_url text check (proof_url is null or proof_url ~* '^https://'),
  visible boolean default true, sort_order int default 0, is_sample boolean
not null default false
);

create table services (
  id uuid primary key default gen_random_uuid(),
  slug text not null unique, name text not null, icon text,
  problem_solved text, description text, deliverables text[] default '{}',
process_steps text[] default '{}',
  starting_price text,                                -- free text e.g.
'From $300' or 'Request a quote'
  timeline_note text, related_project_ids uuid[] default '{}',
  visible boolean default true, sort_order int default 0, is_sample boolean
not null default false
);
create table service_packages (
  id uuid primary key default gen_random_uuid(),
  service_id uuid references services(id) on delete cascade,
  name text not null, price_text text, description text, features text[]
default '{}',
  recommended boolean default false, sort_order int default 0, is_sample
boolean not null default false
);
create table faqs (
  id uuid primary key default gen_random_uuid(),
  question text not null, answer text not null, service_id uuid references
services(id) on delete set null,
  visible boolean default true, sort_order int default 0, is_sample boolean
not null default false
);
```

```
create table testimonials (
  id uuid primary key default gen_random_uuid(),
  quote text not null, author_name text not null, author_role text,
author_company text,
  photo_media_id uuid references media_assets(id) on delete set null,
  permission_confirmed boolean not null default false,
  approval text not null default 'pending' check (approval in
('pending','approved','hidden')),
  project_id uuid references projects(id) on delete set null,
  sort_order int default 0, is_sample boolean not null default false,
  constraint approved_needs_permission check (approval <> 'approved' or
permission_confirmed)
);
```

## 8. SQL — Blog, SEO, Legal

```
create table blog_categories (id uuid primary key default gen_random_uuid(),
name text not null, slug text not null unique, is_sample boolean not null
default false);
create table blog_posts (
  id uuid primary key default gen_random_uuid(),
  slug text not null unique, title text not null, excerpt text, body text,
  category_id uuid references blog_categories(id) on delete set null,
  cover_media_id uuid references media_assets(id) on delete set null,
  tags text[] default '{}', reading_minutes int,
  status content_status not null default 'draft', publish_at timestamptz,
  seo_title text, seo_description text,
  is_sample boolean not null default false,
  created_at timestamptz not null default now(), updated_at timestamptz not
null default now()
);
create table seo_pages (
  route text primary key,                -- '/', '/projects', '/resume',
'/blog', '/privacy'
  title text, description text, og_media_id uuid references media_assets(id)
on delete set null,
  noindex boolean default false, canonical_url text, updated_at timestamptz
default now()
);
create table legal_pages (
```

```
  slug text primary key,                  -- 'privacy'
  title text not null, body text not null, updated_at timestamptz default
now(), is_sample boolean not null default false
);
```

## 9. SQL — Inquiries, Analytics, Security Tables

```
create table inquiries (
  id uuid primary key default gen_random_uuid(),
  name text not null check (char_length(name) between 1 and 80),
  email text not null check (email ~* '^[^@\s]+@[^@\s]+\.[^@\s]+$'),
  service text, budget text, timeline text,
  message text not null check (char_length(message) between 20 and 5000),
  project_slug text,                                  -- when started from a
case study
  status inquiry_status not null default 'new',
  notify_status text not null default 'pending' check (notify_status in
('pending','sent','failed','skipped')),
  consent boolean not null default false,
  ip_hash text,                                       -- salted hash only,
never raw IP
  user_agent_family text,
  is_sample boolean not null default false,
  created_at timestamptz not null default now(), read_at timestamptz
);
create index inquiries_status_idx on inquiries (status, created_at desc);

create table inquiry_notes (
  id uuid primary key default gen_random_uuid(),
  inquiry_id uuid not null references inquiries(id) on delete cascade,
  note text not null, created_at timestamptz default now(), created_by uuid
references auth.users(id)
);

create table analytics_events (
  id bigint generated always as identity primary key,
  occurred_at timestamptz not null default now(),
  event text not null check (event in
('page_view','project_view','cta_click','form_start','form_submit','intro_pla
y','intro_skip','intro_complete')),
  path text, project_slug text, label text, referrer_host text,
```

```
  session_id text,                                    -- random per page
load, memory-only on client
  device text check (device in ('mobile','tablet','desktop')),
  country text
);
create index analytics_time_idx on analytics_events (occurred_at desc);

create table rate_limits (
  key text not null, window_start timestamptz not null, count int not null
default 1,
  primary key (key, window_start)
);

create table audit_log (
  id bigint generated always as identity primary key,
  at timestamptz not null default now(),
  user_id uuid, action text not null,                --
insert|update|delete|login|export|import
  table_name text, row_id text, summary text, diff jsonb
);
```

## 10. Row Level Security (summary + core policies)

Enable RLS on **every** table above (`alter table <t> enable row level security;`). Policy pattern:

```
-- Public read: published content only
create policy projects_public_read on projects for select to anon,
authenticated
  using (is_live(status, publish_at));
create policy projects_admin_all on projects for all to authenticated
  using (is_admin()) with check (is_admin());

-- Public read for always-public singletons / lists
create policy settings_public_read on site_settings for select to anon,
authenticated using (true);
create policy settings_owner_write on site_settings for update to
authenticated using (is_owner()) with check (is_owner());

-- Visible-flag tables (skills, experiences, services, faqs, social_links,
nav_items, ...)
create policy skills_public_read on skills for select to anon, authenticated
```

```
using (visible);
create policy skills_admin_all on skills for all to authenticated using
(is_admin()) with check (is_admin());

-- Testimonials: only approved + permission confirmed
create policy testimonials_public_read on testimonials for select to anon,
authenticated
  using (approval = 'approved' and permission_confirmed);

-- Inquiries / analytics / audit: NO anon policy at all (inserts happen in
Edge Functions with service role)
create policy inquiries_admin_read on inquiries for select to authenticated
using (is_admin());
create policy inquiries_admin_update on inquiries for update to authenticated
using (is_admin()) with check (is_admin());
create policy inquiries_owner_delete on inquiries for delete to authenticated
using (is_owner());
create policy analytics_admin_read on analytics_events for select to
authenticated using (is_admin());
create policy audit_owner_read on audit_log for select to authenticated using
(is_owner());

-- admin_users: owner only
create policy admin_users_owner_all on admin_users for all to authenticated
using (is_owner()) with check (is_owner());
create policy admin_users_self_read on admin_users for select to
authenticated using (user_id = auth.uid());
```

| Table group | anon | editor | owner |
|---|---|---|---|
| Content (projects, skills, services, blog, etc.) | read published/visible | full CRUD | full CRUD |
| site_settings, intro_video, nav, home_sections | read | update (non-danger fields) | full |
| media_assets | read | CRUD | CRUD |
| inquiries, inquiry_notes | none | read/update/notes | + delete |
| analytics_events | none (via function) | read | read + purge |
| audit_log | none | none | read |

| Table group | anon | editor | owner |
|---|---|---|---|
| admin_users | none | read self | full |

Project `view_count` is incremented only via the `track-event` function (security definer RPC `increment_project_view(slug)`), never by direct anon update.

## 11. Storage Buckets and Policies

| Bucket | Public read | Write | Limits / MIME |
|---|---|---|---|
| `media` | yes | admins | ≤ 5 MB; image/webp, image/jpeg, image/png, image/avif, image/svg+xml (sanitized) |
| `intro` | yes | admins | ≤ 50 MB; video/mp4, video/webm, image/webp, text/vtt |
| `documents` | yes | admins | ≤ 10 MB; application/pdf |
| `private-exports` | no | admins | JSON backups, signed URLs only |

Storage policies mirror `is_admin()` for insert/update/delete. File names are generated (`uuid.ext`); original name stored in `media_assets.title`.

## 12. Edge Function Contracts

### `submit-inquiry` (POST, public)

Request:

```
{ "name":"", "email":"", "service":"", "budget":"", "timeline":"",
"message":"", "projectSlug":"", "consent":true, "website":"",
"turnstileToken":"" }
```

`website` is the honeypot (must be empty). Responses:

| Status | Body | When |
|---|---|---|
| 200 | `{ "ok": true, "id": "uuid" }` | Stored (email may still be pending/failed — recorded in `notify_status`) |
| 400 | `{ "ok": false, "code": "VALIDATION", "fields": { "email": "Invalid email" } }` | Schema failure |
| 403 | `{ "ok": false, "code": "CAPTCHA" }` | Turnstile failed |
| 429 | `{ "ok": false, "code": "RATE_LIMIT", "retryAfter": 1800 }` | More than 3 per hour or 10 per day per hashed IP |
| 500 | `{ "ok": false, "code": "SERVER" }` | Unexpected error |

Steps: validate → honeypot (return fake 200 without storing) → Turnstile verify → rate limit using `rate_limits` → insert (service role) → send email via Resend → update `notify_status` → respond.

### `track-event` (POST, public, always returns 204)

Body `{ event, path, projectSlug, label, sessionId, device }`; ignores when `analytics_enabled` is false or DNT is set; whitelists event names; truncates strings; never stores form content or raw IP; calls `increment_project_view` for `project_view`.

### `cleanup-retention` (scheduled daily)

Deletes inquiries older than `inquiry_retention_days`, analytics older than `analytics_retention_days`, expired rate-limit windows; writes an audit entry.

### Public read RPCs (optional, for fewer round-trips)

`get_home_bundle()` returns settings, profile, socials, nav, sections, featured projects, skill groups, services, experiences, approved testimonials, faqs in one JSON payload (respecting RLS via invoker rights).

# 13. Sample (Dummy) Data Plan

All rows below are inserted with `is_sample = true`. The same data lives in `src/seed/*.json` for demo mode and in `supabase/seed.sql` for the real database. **No real brands, people, quotes, links or statistics.** All URL fields are `null` in sample rows (so no fake buttons render).

| Table | Sample rows |
|---|---|
| `profile` | name “Your Name”, title “Creative Developer & AI Designer”, short + long sample bio, 3 values (Quality, Clarity, Curiosity), 3 rotating roles |
| `site_settings` | contact_email `hello@example.com`, availability on |
| `social_links` | GitHub, LinkedIn, Instagram, YouTube, Email (sample URLs `https://example.com/...` flagged sample, shown only in demo) |
| `nav_items`, `home_sections` | Default 6 nav items + CTA; 9 home sections in default order |
| `intro_video` | enabled, `video_*` null, built-in generated poster (gradient) so fallback/poster flow is testable without a real video |
| `categories` | Web Development, AI Projects, UI/UX, Graphic Design, Video Editing, Branding |
| `tags` | React, TypeScript, Figma, Python, Branding, Motion, Video, SEO |
| `projects` | **8 sample projects** (3 featured), spread across all 6 categories, varied tools, 2 with gallery of 4 images, 1 with demo video placeholder, 1 “scheduled”, 1 “draft” (to demo admin states). Titles like “Sample Project — Analytics Dashboard”; text in neutral lorem-style but readable English; results left empty on most to demonstrate hidden sections |
| `skill_groups` / `skills` | 6 groups, ≈ 24 skills (some `is_tool`), each with 1-line context, no percentages |

| Table | Sample rows |
|---|---|
| `experiences` | 3 entries (current + 2 past) with generic roles and sample dates |
| `education`, `certifications`, `achievements` | 1–2 each, clearly generic ("Sample University"); achievements without numeric metrics |
| `services` | 4 services (Web Design & Development, AI Automation, UI/UX Design, Video & Motion) with deliverables and process steps; `starting_price = 'Request a quote'` |
| `service_packages` | 3 sample packages for one service (flagged, hidden unless `pricing_enabled`) |
| `faqs` | 6 generic Q&As |
| `testimonials` | 3 entries with `approval='approved'` **only in demo/seed**; names like "Sample Client A"; flagged sample. Launch Readiness blocks go-live until deleted or replaced |
| `blog_posts` | 3 posts (1 draft), only visible when blog flag on |
| `legal_pages` | Generic privacy policy template with a banner "Review before launch" |
| `seo_pages` | Entries for `/`, `/projects`, `/resume`, `/privacy` |
| `inquiries` | 4 sample inquiries (statuses new/replied/closed/spam) in demo mode only so admin inbox is demonstrable; the production seed file omits them |
| `analytics_events` | Demo mode only: generated 30-day series for charts; production starts empty |

## Sample-data helper functions

```
-- count remaining sample rows per table (dashboard + Sample Data screen)
create or replace function sample_counts() returns table(table_name text, n
bigint)
language plpgsql security invoker as $$
begin
```

```
  if not is_admin() then raise exception 'forbidden'; end if;
  return query select 'projects', count(*) from projects where is_sample
    union all select 'skills', count(*) from skills where is_sample
    union all select 'services', count(*) from services where is_sample
    union all select 'testimonials', count(*) from testimonials where
is_sample
    union all select 'experiences', count(*) from experiences where is_sample
    union all select 'faqs', count(*) from faqs where is_sample
    union all select 'blog_posts', count(*) from blog_posts where is_sample
    union all select 'media_assets', count(*) from media_assets where
is_sample
    union all select 'inquiries', count(*) from inquiries where is_sample;
end $$;

-- owner-only purge, called by the Delete-all-sample-data button after type-
to-confirm
create or replace function purge_sample_data() returns void language plpgsql
security invoker as $$
begin
  if not is_owner() then raise exception 'forbidden'; end if;
  delete from testimonials where is_sample;   delete from project_media where
project_id in (select id from projects where is_sample);
  delete from projects where is_sample;       delete from skills where
is_sample;     delete from skill_groups where is_sample;
  delete from experiences where is_sample;    delete from education where
is_sample;  delete from certifications where is_sample;
  delete from achievements where is_sample;   delete from service_packages
where is_sample; delete from faqs where is_sample;
  delete from services where is_sample;       delete from blog_posts where
is_sample; delete from inquiries where is_sample;
  delete from media_assets where is_sample;   delete from social_links where
is_sample;
  insert into audit_log(user_id, action, summary) values (auth.uid(),
'delete', 'purge_sample_data');
end $$;
```

Singleton tables ( `profile` , `intro_video` , `site_settings` ) are not deleted: editing them with real values and ticking “This is real content” sets `is_sample = false` . The app treats any row where an edit changed the main text fields as a prompt to clear the flag.

## 14. Launch Readiness Query (dashboard)

Ready when all are true: `profile.is_sample = false`; `site_settings.contact_email` not `hello@example.com`; at least 3 real published projects; CV document exists in `documents`; intro video set or intro disabled; no sample testimonials visible; `seo_pages` filled for `/`; email notify test succeeded at least once; `sample_counts()` total = 0 (or owner clicked “Launch with sample content anyway”).

## 15. Backup and Export Format

```
{ "schema": 1, "exportedAt": "ISO-8601", "tables": { "projects": [...],
"skills": [...], "...": [...] }, "media": [ { "id":"", "publicUrl":"",
"alt":"" } ] }
```

Export excludes `auth` data, `admin_users` and `audit_log`. Import validates `schema`, shows counts per table, snapshots current data first, upserts by `id`, and rejects unknown tables.

## 16. Migration Order

1. `0001_init.sql` — types, functions, tables (sections 3–9).
2. `0002_rls.sql` — enable RLS + policies (§10) + storage policies (§11).
3. `0003_triggers.sql` — `set_updated_at`, audit triggers, project revision trigger, slug uniqueness helpers.
4. `0004_functions.sql` — `sample_counts`, `purge_sample_data`, `increment_project_view`, `get_home_bundle`.
5. `seed.sql` — sample data (§13).
6. Create first Owner: sign up in Supabase Auth, then run `insert into admin_users(user_id, role) values ('<auth user id>', 'owner');` once.

## 17. Backend Acceptance Tests

- Anonymous client **cannot** select drafts, scheduled-in-future rows, unapproved testimonials, inquiries, analytics, audit log or admin users.
- Anonymous client **cannot** insert/update/delete anything directly.

- Editor can edit content but not `admin_users`, owner-only settings fields, purge or delete inquiries.
- `submit-inquiry` rejects bad email, short message, bad Turnstile, filled honeypot (no storage), and the 4th request within an hour from the same hashed IP.
- Replacing a media file keeps URL stable; deleting an in-use media row is blocked.
- `purge_sample_data()` leaves real rows untouched.
- Service-role and email keys never appear in the client bundle or in responses.